Learning to Board

Technical reference for presentation. An RL gate controller that chooses **boarding order** for a tick-based airplane cabin simulator. Every figure in this document was measured by running the simulator when the document was generated, or read directly from the code and policy metadata.

Item	Value
Shipped agent	`rl_parity_mid` — tabular REINFORCE with a paired baseline
Source of truth	The Python simulator. The front end only replays recorded trajectories.
Evaluation seeds	500–529 (30 seeds), never used in training
Training seeds	1–48 (48 seeds), cycled
Validation seeds	300–319 (20 seeds), used only to pick between restarts
Discount γ	1.0 (undiscounted) for every shipped result
Revision	cd09354

Headline, stated honestly. The agent beats every fixed heuristic on the cabin it was trained on, and loses to Steffen on cabins it was not trained on. Both results are in this document. See “Negative results”.

1. Architecture

Three layers with a strict one-way dependency: the simulator knows nothing about the API, and the API knows nothing about the front end's rendering.

Layer	Module	Responsibility
Simulation	`boarding/simulator.py`	Tick-based cabin model. Aisle occupancy, walking, stowing, seat shuffles, blocking. Produces a trajectory log.
Cabin presets	`boarding/config.py`	Row/column/door/deck geometry for 7 cabins.
Heuristics	`boarding/strategies.py`	The five fixed baselines, expressed as passenger selectors.
MDP	`boarding/envs.py`	Wraps the simulator as a Gymnasium environment: state encoding, action masking, reward.
Agents	`boarding/agents/`	Bandits, exact DP, tabular MC/TD, linear FA, and the shipped policy gradient.
API	`api/main.py`	FastAPI. Validates with Pydantic, runs the simulator, returns a trajectory as JSON. Background training jobs.
Front end	`frontend/src/`	React + TypeScript + Vite. Replays the recorded trajectory in an SVG cabin. Invents no simulation events.

Why trajectory replay rather than live streaming

The backend runs the whole episode and returns a delta-encoded frame list. The front end scrubs through it. This means playback speed, pause, replay and the timeline scrubber cost nothing and never re-run the simulation, and it guarantees that what is on screen is exactly what the Python engine produced.

2. The simulator

Discrete time. One **tick** is the atomic unit; all reported times are in ticks, not seconds, because the tick-to-second mapping is a calibration choice and was not calibrated against real boarding data.

Per-tick update order

Each tick the simulator (1) asks the controller whether to release a waiting passenger into the aisle, (2) decrements stowing and seat-shuffle timers, (3) moves every in-aisle passenger one step toward their row if and only if the next aisle cell is free, and (4) seats anyone who has arrived and finished their delays.

Constraints enforced by construction

- A single aisle cell holds at most one passenger, so passengers cannot pass through one another. A slow passenger blocks everyone behind them.
- A passenger only ever moves toward their assigned row.
- A seat is occupied by exactly one passenger; seat assignment is a permutation and is never optimised — only the *order* is.
- A passenger is not counted as seated until they have reached their row *and* finished stowing *and* finished any seat-shuffle delay.
- Boarding time is read from the simulator clock when the last passenger sits, never estimated.

Timing parameters (defaults from `SimConfig`)

Item	Value
`walk_ticks_per_row`	1 — ticks to advance one row along the aisle
`stow_ticks`	6 — ticks to stow a carry-on, blocking the aisle
`no_bag_ticks`	2 — ticks for a passenger with no carry-on
`shuffle_ticks`	4 — ticks per already-seated passenger who must stand to let someone in
`luggage_fraction`	0.7 — share of passengers carrying a bag
`max_ticks`	20000 — safety cap; an episode that hits this is a bug, not a result
`seed`	42 — controls manifest, luggage and arrival order

The seat-shuffle rule is the main source of interference and therefore the main thing an agent can learn to avoid: boarding a window passenger after the aisle passenger in the same row forces the aisle passenger to stand up.

Reproducibility

A given (aircraft, seed, arrival, luggage fraction) tuple always produces the same passenger manifest, the same seat assignment and the same luggage flags. Strategy comparison therefore holds the manifest fixed and varies only the boarding order, which is the only fair way to compare.

3. Cabins and strategies

Cabin	Name	Rows	Seats	Layout
`atr72`	ATR 72 (2-2, regional)	18	72	2-2
`a320`	A320 / 737 (3-3)	30	180	3-3
`b787`	Boeing 787 (3-3-3)	36	324	3-3-3
`b777`	Boeing 777 (3-4-3)	42	420	3-4-3
`a380`	Airbus A380 (2 decks)	74	680	3-4-3 / 2-4-2
`mid`	Mid cabin (10 rows, 3-3)	10	60	3-3
`tiny`	Tiny cabin (4 rows, 2-2)	4	16	2-2

`mid` and `tiny` are not real aircraft. `tiny` is small enough for exact dynamic programming and for hand-checked tests; `mid` is the training cabin, chosen because an episode is short enough to run tens of thousands of times.

The five fixed baselines

Strategy	Rule
`random`	Uniformly random order. The control condition.
`back_to_front`	Board the rows furthest from the door first, in blocks. Widely used by airlines and, as the measurements below show, the *worst* of the five.
`wilma`	Window, then middle, then aisle, across the whole cabin. Removes seat shuffles entirely.
`outside_in`	WILMA refined with a distance term so that, within a seat-type sweep, far rows go first.
`steffen`	Alternating rows within a window/middle/aisle sweep, so consecutive passengers stow two rows apart and do not block each other. The strongest published heuristic.

Implementation decision. “Steffen” here is the practical alternating-row interpretation, not the strict perfect-ordering version, because the strict version assumes passengers queue in an exact sequence that no real gate can enforce. This is stated because it affects the comparison.

4. The MDP

The agent is a **gate controller**. It does not move passengers and it does not assign seats. Once per tick it chooses which *group* of waiting passengers to release next. Movement and aisle interaction are the simulator's job. These three things — arrival pattern, release decision, and movement — are deliberately separate.

Action space

Item	Value
Shape	distance band × seat type × row parity
Size	4 × 3 × 2 = **24** actions
Distance bands	4, measured as distance from the active door, so the encoding works for front, rear and both-door boarding
Seat types	window, middle, aisle
Row parity	even / odd row index — this is what lets the agent discover a Steffen-like alternating pattern
Masking	Groups with no waiting passenger are masked out; choosing an unavailable group is penalised so the policy learns to avoid it.

Why groups and not individual passengers. Choosing among N passengers gives an action space that grows with the aircraft and a policy that cannot transfer between cabins. Choosing among 24 fixed group descriptors keeps the action space constant and size-independent.

State space

Item	Value
Encoding	(boarding progress bin, last band, last seat type, last parity)
Progress bins	`PROGRESS_BINS = 12` — fraction of passengers seated, discretised
Why “last action” is in the state	It is what makes alternation expressible: to avoid releasing two adjacent rows in a row, the policy must know what it just released.
Upper bound	12 × 5 × 4 × 3 = 720 states
Actually visited	289

Design decision. Group availability is handled by *action masking*, not by state bits. An earlier version encoded which groups were non-empty; it produced roughly 19,000 states and learned worse than random, because almost every state was visited once.

Reward

```
reward = −ticks_elapsed − block_penalty × blocks − invalid_penalty
```

- **−1 per tick** is the whole objective: maximising return is exactly minimising boarding time. No reward shaping is needed and none is used.
- **block_penalty** defaults to `0.0`, i.e. it is *off*. It is exposed because the specification asks for an optional congestion penalty, but shipping it on would mean optimising a different objective than the one being reported.
- **invalid_penalty** discourages selecting a masked-out group. It is a training-stability device, not part of the objective.

Episode termination

An episode ends when the last passenger is seated. There is no step limit other than the `max_ticks` safety cap; because every released passenger eventually sits, termination is guaranteed.

5. Training

Algorithm: tabular REINFORCE (Monte-Carlo policy gradient) with a paired baseline. The policy is a softmax over a table of action preferences indexed by the discrete state.

Why policy gradient rather than Q-learning

This was determined empirically, and the losing runs are kept in the repository as evidence. Value-based methods struggle here because the reward is entirely terminal in character — the cost of releasing a group appears 20–40 ticks later as an aisle jam — so one-step TD cannot assign credit, and the action-value differences between good and bad groups are small relative to the noise in a single episode. A policy gradient with a paired baseline sidesteps this by comparing *whole episodes on the same seed*.

The paired baseline (the single most important detail)

Boarding time varies more between *seeds* than between *policies*. Comparing an episode on seed 7 against a running average over other seeds is mostly measuring seed difficulty, not policy quality. Instead, each update compares an episode to a decayed baseline *for that same seed*, so the manifest difficulty cancels out. Without this the agent does not learn.

Hyperparameters

Item	Value
Learning rate α	0.25
Discount γ	1.0, undiscounted — see note below
Batch size	16 episodes per gradient update
Entropy bonus	0.01 in the CLI trainer, 0.0 from the API
Baseline decay	0.3
Episodes	12000
Restarts	6
Exploration	Softmax sampling. There is no ε-greedy; exploration is intrinsic to the stochastic policy and decays as preferences sharpen.

On $\gamma = 1$. The objective is total boarding time, a finite undiscounted sum. Any $\gamma < 1$ would optimise a different objective and would under-weight exactly the delayed congestion the agent needs to learn about. The implementation gives every step in an episode the same advantage, which *is* $\gamma = 1$ structurally. A γ field was removed from the training API rather than left as a knob that silently changes what is being optimised.

Restarts and model selection

Policy gradient on a tabular softmax is sensitive to initialisation. The trainer runs several independent restarts and selects between them on the **validation seeds (300–319)**, which are disjoint from both the training seeds and the evaluation seeds. The selected policy is then scored once on the evaluation seeds. This three-way split is what makes the reported number an honest estimate rather than a best-of-N on the test set.

Validation scores of the restarts for the shipped policy: 183.2, 154.0, 175.3, 178.2, 169.4, 182.8 ticks. The best was selected.

Commands

```
# train the shipped agent (mid cabin, with row parity)
PYTHONPATH=. .venv/bin/python scripts/train_parity.py \
  --cabin mid --episodes 10000 --restarts 3 --save-as rl_parity_mid

# compare a policy against every baseline across cabins
PYTHONPATH=. .venv/bin/python scripts/probe_rl_vs_baselines.py rl_parity_mid
```

6. Evaluation method

The claim being tested is: *does the learned release rule board a cabin faster than the best fixed heuristic, on manifests it has never seen?*

Control	How it is handled
Same manifests	Every strategy and the agent are run on the identical (aircraft, seed) pairs, so passenger count, seat assignment, luggage flags and arrival order are byte-identical across conditions.
Held-out seeds	Evaluation uses seeds 500–529. Training used 1–48 and model selection used 300–319. No overlap.
Repeats	30 independent seeds per condition. Mean and standard deviation are both reported, because a mean alone hides whether a 3-tick gap is signal.
Determinism	The policy is evaluated greedily (arg-max), so a given (policy, seed) pair always gives the same number.
Off-distribution test	The same policy is run on cabins it was never trained on. This is reported even though it is unfavourable.

Metrics and their units

Metric	Unit	Definition
Boarding time	ticks	Simulator clock when the final passenger is seated. The primary metric.
Mean / Std	ticks	Across the 30 evaluation seeds. Std is the population standard deviation.
Best / Worst	ticks	The easiest and hardest seed, to show spread.
Average wait	ticks	Mean per-passenger time between being released and being seated. A secondary diagnostic: a policy can shorten total time while lengthening individual waits.
Blocked count	passenger-ticks	Number of (passenger, tick) pairs where a passenger could not advance. The congestion measure.

7. Measured results

Measured when this PDF was generated, 30 seeds per row, lower is better.

mid cabin (training cabin)

Strategy	Mean	Std	Best	Worst	Avg wait
rl_parity_mid	154.0	7.9	140	169	12.4
rl_agent_mid	157.1	9.3	135	176	13.8
wilma	159.0	10.9	138	181	12.9
steffen	159.1	10.1	142	184	13.0
outside_in	161.0	11.7	143	183	14.7
rl_agent	164.8	9.1	143	182	14.4
random	221.5	18.7	178	250	17.7
back_to_front	265.3	15.4	237	294	24.9

All figures in ticks.

A320 (not trained on)

Strategy	Mean	Std	Best	Worst	Avg wait
steffen	379.3	13.8	349	406	26.4
wilma	383.7	13.7	355	419	26.7
outside_in	386.9	12.7	362	421	31.4
rl_agent_mid	392.1	15.7	363	430	32.3
rl_parity_mid	394.0	13.6	371	434	28.8
rl_agent	407.0	14.1	369	435	34.0
random	493.5	26.7	433	559	33.3
back_to_front	615.6	30.5	566	691	52.7

All figures in ticks.

How to read this

On the training cabin the agent is at or ahead of the best heuristic. On the A320 it is not. That gap is the honest finding of the project: the policy learned a release rule that is *tuned to a cabin geometry*, not a universal boarding law.

8. Negative results (kept deliberately)

These are included because a presentation that reports only the favourable run is not a result. Each of these cost real compute and each one shaped the final design.

Attempt	Outcome	What it means
Watkins' Q(λ), SARSA(λ), Monte-Carlo control	All slower than the fixed heuristics	Value-based tabular control does not solve this credit-assignment problem at this sample budget. Kept in `artifacts/policies/` as evidence; withheld from the UI dropdown so they do not masquerade as “the agent”.
Encoding group availability in the state	~19,000 states, learned worse than random	State explosion. Replaced with action masking.
Training a parity agent directly on the A320	6k episodes: 426.5 ticks. 24k episodes: 421.0 ticks. Steffen: 379.3	Quadrupling the sample budget recovered ~5 ticks of a ~47-tick gap, so this is a **representation limit, not an undertraining one**: the band/parity encoding does not carry enough information on a longer cabin. Both policies were **deleted rather than shipped**.
Discount sweep on the tiny cabin	Every γ from 0.5 to 1.0 returned the same optimal policy	The tiny cabin's horizon is too short for the discount to bite. Reported as a null result rather than dressed up.

What is explicitly not claimed. That the learned policy is optimal; that it generalises across aircraft; that ticks correspond to any particular number of seconds; or that these results transfer to real boarding, which involves passenger behaviour this simulator does not model.

9. API surface

Endpoint	Method	Purpose
`/api/health`	GET	Liveness.
`/api/options`	GET	Everything the UI is allowed to offer: aircraft, gates, arrivals, strategies, policies. The UI renders this rather than hardcoding, so an option cannot appear in the UI without the backend supporting it.
`/api/simulate`	POST	Run one episode. Returns stats plus a delta-encoded trajectory.
`/api/compare`	POST	Run several strategies over several seeds on identical manifests.
`/api/train`	POST	Start a background training job. Returns a job id immediately.
`/api/train/{job_id}`	GET	Poll job status, progress and errors.
`/api/policies`	GET	Saved, loadable policies.

Validation is Pydantic; errors come back as a structured `{"error": {code, message, detail}}` envelope, including for unhandled exceptions, so the front end never has to parse a plain-text 500.

Options withheld from the UI

Three inputs are accepted by the API but not offered in the interface: the `tiny` cabin (a test fixture, not an aircraft), the `queued` arrival pattern (the MDP does not read the arrival pattern, so comparisons under it are not meaningful), and a training `gamma` (removed; see the γ=1 note). Scripts and tests still use the first two.

10. Testing

Suite	Tests	Covers
`tests/test_agents.py`	15	State encoding, action masking, reward sign, terminal conditions, policy save/load round-trip, backward-compatible loading of 12-action policies.
`tests/test_api.py`	32	Request validation, error envelopes, trajectory serialisation, that every advertised option actually runs, and that withheld options stay withheld.
`tests/test_simulator.py`	19	Movement, aisle blocking, luggage delays, seat shuffles, completion time, determinism under a fixed seed, hand-computed tiny-cabin scenarios.
Total	**66**	

Plus front-end tests under `frontend/src/` covering trajectory decoding and rendering against real API-shaped fixtures.

The hand-verifiable test

The most important test is the smallest: a cabin small enough that the expected boarding time can be worked out with pen and paper, asserted exactly. If the walking, stowing and blocking arithmetic is wrong, that test fails before any RL result can be trusted.

Commands

```
PYTHONPATH=. .venv/bin/python -m pytest -q # 66 Python tests
cd frontend && npm test # front-end tests
cd frontend && npm run typecheck # TypeScript
cd frontend && npm run build # production build
```

11. Running it

```
# terminal 1 — backend
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
PYTHONPATH=. .venv/bin/uvicorn api.main:app --reload --port 8000

# terminal 2 — front end
cd frontend && npm install && npm run dev

# then open http://localhost:5173
```

Requires Python 3.9+ and Node 20+. The Vite dev server binds to `localhost`; use that hostname rather than 127.0.0.1.

A demonstration that exercises everything

- Pick the `mid` cabin, front door, random arrival, seed 500. Run `steffen` and watch the cabin fill.
- Switch the strategy to `rl_parity_mid` on the same seed. The manifest is identical, so any difference is the policy.
- Scrub the timeline back and forth — it is replaying a recorded trajectory, not re-simulating.
- Open **Compare** and run all five baselines plus the agent over a seed range to see means and spread rather than one lucky run.
- Switch the cabin to the A320 and re-run the comparison to show the generalisation gap.
- Open **Method & parameters** for the live hyperparameters of whichever policy is selected.
- Open **Train** to start a short run and show that training is a real background job with progress, not a canned animation.

12. Likely questions

Is the agent actually learning, or is it just picking a heuristic?

It is learning a state-conditioned release rule. The policy table has 289 visited states with distinct action distributions; it is not a lookup of one of the five baselines. The strongest evidence is that it beats all five on the training cabin while being worse off-distribution — a heuristic selector could not do that.

Why is the reward just −1 per tick?

Because that is literally the objective. Any shaping term would optimise something other than boarding time, and would then need justifying as a proxy. The optional congestion penalty exists but is off by default.

Why $\gamma = 1$?

Finite undiscounted episodic return. Discounting would under-weight the delayed congestion that is the entire difficulty of the problem.

How do you know you are not overfitting the evaluation seeds?

Three disjoint seed pools: training 1–48, restart selection 300–319, reporting 500–529. Model selection never touches the reporting pool.

Why does it lose on the A320?

The state uses distance *bands* and row parity. Band boundaries and the density of passengers per band change with cabin length, so a preference tuned on 10 rows does not mean the same thing on a longer cabin. Training directly on the A320 was attempted at two sample budgets; quadrupling the episodes closed only about a tenth of the gap to Steffen, which points at the state representation rather than undertraining. Both runs were deleted rather than shipped.

Why not use deep RL?

The state space is a few hundred discrete states. A neural network would add variance, non-determinism and a dependency without addressing the actual bottleneck, which is credit assignment over a long horizon.

What is one tick in seconds?

Undefined, deliberately. The timing constants were not calibrated against real boarding data, so quoting seconds would be a fabricated precision. All comparisons are relative and in ticks.

What is the biggest weakness?

Generalisation across cabin geometry, followed by the fact that passengers are modelled as compliant — they board exactly when released and walk at a fixed rate. Real boarding has late arrivals, groups refusing to split, and gate-agent discretion.